

Routes API : 接口文档

1. 通过定义路由规则来匹配客户端的请求
2. 根据匹配结果加载并执行相应的插件
3. 并把请求转发到指定 Upstream

获取资源: 所有资源列表

允许已授权的用户通过此接口获取资源列表。

- URL: `/edge/admin/routes`
- Method: `GET`
- 需要登录: `是`
- 需要鉴权: `是`
- 输入数据: `通过接口方式传输`
- 输出数据: `json格式返回`
- 接口调用出于安全考虑带密码认证, 需要在请求头中添加 `X-API-KEY` 属性

请求参数

无

请求示例

```
curl -L -X GET 'http://192.168.100.233:9990/edge/admin/routes' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应

条件: 请求参数合法, 并且用户身份校验通过。

状态码: `200 OK`

响应示例: 响应会返回所有资源

```
{
  "header": {
    "revision": 32
  },
  "node": {
    "nodes": [
      {
        "value": {
          "status": 1,
          "uri": "/uuuuu",
          "desc": "ssasdadsdadsssss",

```

```

        "upstream_id": "6ae0082aece8474292d44e626f837bc4",
        "priority": 0,
        "create_time": 1675063448,
        "id": "ed3e5150c2984ba39feb1412cfb7ec1d",
        "name": "sdsdsaweqweqe",
        "update_time": 1675063448
    },
    "createdIndex": 0,
    "key": "\\search\\routes\\ed3e5150c2984ba39feb1412cfb7ec1d",
    "modifiedIndex": 0
},
{
    "value": {
        "status": 1,
        "name": "çÿ¥è¬†â°“è·¬ç”±”,
        "desc": "sssssss",
        "upstream_id": "6ae0082aece8474292d44e626f837bc4",
        "priority": 0,
        "create_time": 1675041718,
        "id": "cf02da9b54b841fc9e6f42cfef2230f2",
        "uris": [
            "\\jy",
            "\\log"
        ],
        "update_time": 1675042449
    },
    "createdIndex": 0,
    "key": "\\search\\routes\\cf02da9b54b841fc9e6f42cfef2230f2",
    "modifiedIndex": 0
},
{
    "value": {
        "upstream": {
            "nodes": {
                "127.0.0.1:8111": 1
            },
            "scheme": "http",
            "type": "roundrobin",
            "hash_on": "vars",
            "pass_host": "pass"
        },
        "create_time": 1675839823,
        "uri": "\\hello1",
        "id": "3001",
        "priority": 0,
        "status": 1,
        "update_time": 1675841122
    },
    "createdIndex": 0,
    "key": "\\search\\routes\\3001",
    "modifiedIndex": 0
},
{
    "value": {
        "service_id": "431261175397745549",
        "status": 1,
        "uri": "\\proxy\\v2\\*",
        "desc": "xxxxx",

```

```

    "labels":{
      "API_VERSION":"V2",
      "L111":"x111"
    },
    "id":"433437201016030093",
    "plugins":{
      "proxy-rewrite":{
        "regex_uri":[
          "^\\proxy\\/(.*)$",
          "\\$1"
        ]
      }
    },
    "methods":[
      "GET",
      "POST",
      "PUT",
      "DELETE",
      "PATCH",
      "HEAD",
      "OPTIONS",
      "CONNECT",
      "TRACE"
    ],
    "name":"RS-Proxy"
  },
  "createdIndex":0,
  "key":"\\search\\routes\\433437201016030093",
  "modifiedIndex":0
},
{
  "value":{
    "methods":[
      "GET",
      "POST",
      "PUT",
      "DELETE",
      "PATCH",
      "HEAD",
      "OPTIONS",
      "CONNECT",
      "TRACE"
    ],
    "status":0,
    "name":"R-nacos",
    "id":"431260676158129037",
    "plugins":{
      "proxy-rewrite":{
        "uri":"\\actuator"
      }
    },
    "uri":"\\nacos\\/*",
    "upstream_id":"431260237702366093"
  },
  "createdIndex":0,
  "key":"\\search\\routes\\431260676158129037",
  "modifiedIndex":0
},

```

```

{
  "value":{
    "methods":[
      "GET",
      "POST",
      "PUT",
      "DELETE",
      "PATCH",
      "HEAD",
      "OPTIONS",
      "CONNECT",
      "TRACE"
    ],
    "status":1,
    "name": "R-proxy",
    "id": "431260450521351053",
    "plugins":{
      "proxy-rewrite":{
        "regex_uri":[
          "^\\proxy/(.*)$",
          "\\$1"
        ]
      }
    },
    "uri": "\\proxy/*",
    "upstream_id": "431253674958783373"
  },
  "createdIndex":0,
  "key": "\\search/routes/431260450521351053",
  "modifiedIndex":0
},
{
  "value":{
    "id": "4343434343",
    "upstream":{
      "type": "roundrobin",
      "nodes":{
        "127.0.0.1:8111":1
      }
    },
    "uri": "\\hello",
    "upstream_id": "433437599709791117"
  },
  "createdIndex":0,
  "key": "\\search/routes/4343434343",
  "modifiedIndex":0
}
],
"createdIndex":0,
"dir": true,
"key": "\\search/routes",
"modifiedIndex":0
},
"action": "get",
"count":7
}

```

响应 Body 体字段解释

见 Body 字段解释

获取资源: 指定 ID 资源列表

允许已授权的用户通过此接口根据 id 获取资源列表。

- URL: /edge/admin/routes/{id}
- Method: GET
- 需要登录: 是
- 需要鉴权: 是
- 输入数据: 通过接口方式传输
- 输出数据: json格式返回
- 接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性

请求参数

参数	类型	方式
id	String	通过 PATH URI 方式获得

请求示例

请求 id 为 3001 的资源

```
curl -L -X GET 'http://192.168.100.73:9990/edge/admin/routes/3001' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应

- 条件:** 请求参数合法，并且用户身份校验通过。
- 状态码:** 200 OK
- 响应示例:** 响应会返回请求 id 对应的资源

```
{
  "action": "get",
  "node": {
    "modifiedIndex": 0,
    "createdIndex": 0,
    "key": "\\search/routes/3001",
    "value": {
      "status": 1,
      "uri": "\\hello",
    }
  }
}
```

```
    "id": "3001",
    "update_time": 1675905313,
    "upstream": {
      "scheme": "http",
      "type": "roundrobin",
      "pass_host": "pass",
      "hash_on": "vars",
      "nodes": {
        "127.0.0.1:8111": 1
      }
    },
    "create_time": 1675905156,
    "priority": 0
  }
},
"header": {
  "revision": 8
},
"count": 1
}
```

响应 Body 体字段解释

见 Body 字段解释

错误响应

条件： 请求 id 对应的资源不存在

状态码： 404 Not Found

响应示例：

```
{
  "action": "get",
  "header": {
    "revision": 8
  },
  "message": "Key not found"
}
```

创建资源: 根据指定 ID 创建

允许已授权的用户通过此接口根据 id 创建资源。

- URL: `/edge/admin/routes/{id}`
- Method: `PUT`
- 请求Body: 资源数据, json 格式
- 输出数据: json格式返回
- 接口调用出于安全考虑带密码认证, 需要在请求头中添加 X-API-KEY 属性
- 需要登录: 是
- 需要鉴权: 是

请求参数

参数	类型	方式
id	String	通过 PATH URI 方式获得

请求 Body 体字段

见 Body 字段解释

请求示例

设置 id 为 3001 的资源

```
curl -L -X PUT 'http://192.168.100.73:9990/edge/admin/routes/3001' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e' -H 'Content-Type: application/json' -d '{
  "uri": "/hello1",
  "upstream": {
    "nodes": {
      "127.0.0.1:8111": 1
    },
    "type": "roundrobin"
  }
}'
```

成功响应

条件: 请求参数合法, 并且用户身份校验通过。

状态码: `200 OK`

响应示例: 响应会返回该 id 对应的资源内容

```
{
  "action": "set",
  "header": {

  },
  "node": {
```

```
    "value":{
      "status":1,
      "uri":"/hello1",
      "id":"3001",
      "update_time":1675924791,
      "create_time":1675905156,
      "priority":0,
      "upstream":{
        "scheme":"http",
        "type":"roundrobin",
        "pass_host":"pass",
        "hash_on":"vars",
        "nodes":{
          "127.0.0.1:8111":1
        }
      }
    },
    "key":"/search/routes/3001"
  }
}
```

响应 Body 体字段解释

见 Body 字段解释

创建资源: 自动生成 ID

允许已授权的用户通过此接口创建资源, ID 由后台服务自动生成。

- URL: `/edge/admin/routes`
- Method: `POST`
- 请求 Body: `资源数据, json 格式`
- 输出数据: `json格式返回`
- 接口调用出于安全考虑带密码认证, 需要在请求头中添加 X-API-KEY 属性
- 需要登录: `是`
- 需要鉴权: `是`

请求参数

无

请求 Body 体字段

见 Body 字段解释

请求示例

以 POST 方式创建资源, 系统会自动生成 ID


```
curl -L -X POST 'http://192.168.100.73:9990/edge/admin/routes' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e' -H 'Content-Type: application/json' --data '{
  "uri": "/hi",
  "upstream": {
    "nodes": {
      "127.0.0.1:8111": 1
    },
    "type": "roundrobin"
  }
}'
```

成功响应

条件： 请求参数合法，并且用户身份校验通过。

状态码： 200 OK

响应示例： 响应会返回创建的资源明细

```
{
  "action": "create",
  "header": {
  },
  "node": {
    "value": {
      "status": 1,
      "uri": "/hi",
      "create_time": 1675925790,
      "update_time": 1675925790,
      "id": "ea6d592cdb4c4b168635ae4c12c50288",
      "upstream": {
        "scheme": "http",
        "type": "roundrobin",
        "pass_host": "pass",
        "hash_on": "vars",
        "nodes": {
          "127.0.0.1:8111": 1
        }
      },
      "priority": 0
    },
    "key": "/search/routes/ea6d592cdb4c4b168635ae4c12c50288"
  }
}
```

响应 Body 体字段解释

见 Body 字段解释

删除资源: 根据指定 ID 删除

允许已授权的用户通过此接口根据 id 删除资源。

- URL: /edge/admin/routes/{id}
- Method: DELETE
- 输出数据: json格式返回
- 接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性
- 需要登录: 是
- 需要鉴权: 是

请求参数

参数	类型	方式
id	String	通过 PATH URI 方式获得

请求示例

删除 id 为 ea6d592cdb4c4b168635ae4c12c50288 的资源

```
curl -L -X DELETE 'http://192.168.100.73:9990/edge/admin/routes/ea6d592cdb4c4b168635ae4c12c50288' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应

条件: 请求参数合法，并且用户身份校验通过。

状态码: 200 OK

响应示例: 响应会返回该 id 对应的资源内容

```
{
  "deleted": "1",
  "key": "\search/routes/ea6d592cdb4c4b168635ae4c12c50288",
  "action": "delete",
  "header": {

  },
  "node": {

  }
}
```

响应 Body 体字段解释

见 Body 字段解释

更新资源: 更新全部属性

允许已授权的用户通过此接口根据 id 创建资源。

- URL： /edge/admin/routes/{id}
- Method： PATCH
- Body： 资源数据, json 格式
- 输出数据： json格式返回
- 接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性
- 需要登录： 是
- 需要鉴权： 是

请求参数

参数	类型	方式
id	String	通过 PATH URI 方式获得

请求 Body 体字段

见 Body 字段解释

请求示例

更新 id 为 3001 的资源

```
curl -L -X PATCH 'http://192.168.100.73:9990/edge/admin/routes/3001' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e' -H 'Content-Type: application/json' --data '{
  "uri": "/hello2",
  "upstream": {
    "nodes": {
      "127.0.0.1:8112": 1
    },
    "type": "roundrobin"
  }
}'
```

成功响应

条件：请求参数合法，并且用户身份校验通过。

状态码： 200 OK

响应示例：响应会返回该 id 对应的资源内容

```
{
  "action": "compareAndSwap",
  "header": {
    "revision": 12
  }
}
```

```
{
  "succeeded":true,
  "node":{
    "value":{
      "status":1,
      "uri":"\\/hello2",
      "id":"3001",
      "update_time":1675926936,
      "upstream":{
        "scheme":"http",
        "type":"roundrobin",
        "pass_host":"pass",
        "hash_on":"vars",
        "nodes":{
          "127.0.0.1:8112":1,
          "127.0.0.1:8111":1
        }
      }
    },
    "create_time":1675905156,
    "priority":0
  },
  "key":"\\/search\\/routes\\/3001"
}
```

响应 Body 体字段解释

见 Body 字段解释

更新资源: 更新部分属性

允许已授权的用户通过此接口根据 id 创建资源。

- URL: `/edge/admin/routes/{id}/{path}`
- Method: `PATCH`
- Body: 资源数据, 数据类型与path类型相同
- 输出数据: `json格式返回`
- 接口调用出于安全考虑带密码认证, 需要在请求头中添加 X-API-KEY 属性
- 需要登录: `是`
- 需要鉴权: `是`

请求参数

参数	类型	方式
id	String	通过 PATH URI 方式获得

请求 Body 体字段

见 Body 字段解释

请求示例

更新 id 为 3001 的资源, 只更改 uri 属性

由于 uri 属性为字符串类型, 所以 body 中需要传入字符串

```
curl -L -X PATCH 'http://127.0.0.1:9990/edge/admin/routes/3001/uri' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e' -H 'Content-Type: application/json' --data '"/helloworld"'
```

成功响应

条件: 请求参数合法, 并且用户身份校验通过。

状态码: 200 OK

响应示例: 响应会返回该 id 对应的资源内容

```
{
  "action": "compareAndSwap",
  "header": {
    "revision": 14
  },
  "succeeded": true,
  "node": {
    "value": {
      "status": 1,
      "uri": "/helloworld",
      "id": "3001",
      "update_time": 1675928166,
      "priority": 0,
      "create_time": 1675905156,
      "upstream": {
        "scheme": "http",
        "type": "roundrobin",
        "pass_host": "pass",
        "hash_on": "vars",
        "nodes": {
          "127.0.0.1:8112": 1,
          "127.0.0.1:8111": 1
        }
      }
    }
  },
  "key": "/search/routes/3001"
}
```

响应 Body 体字段解释

见 Body 字段解释

Body 字段解释

字段	解释	示例	限制
uri	地址	"/hello"	不能与 uris 一起使用
uris	地址	["/hello", "/world"]	多个uri, 不能与 uri 一起使用
upstream	Upstream 配置信息, 独立于 Upstreams列表外		
upstream_id	指定使用Upstreams列表中的已启用的upstream id		
service_id	指定使用Services例表中的已启用的Service id		
plugins	Plugin配置	"plugins": { "proxy-rewrite": { "regex_uri": ["^/proxy/(.*)\$", "/\$1"] } }	
plugin_config_id	指定使用 plugin_configs 配置列表里的 id 项		与plugins配置合并, 如果有相同的插件, 取 id 指定的配置
name	标识路由名称		
desc	标识描述信息。		
host	当前请求域名	"foo.com"	不能与 hosts 一起使
hosts	多个host	["foo.com", "*.bar.com"]	不能与 host 一起使用
remote_addr	客户端请求 IP 地址	"192.168.1.0/24"	IP地址例如: 192.168.1.101、 192.168.1.102 以及CIDR 格式的格式,例如: 192.168.1.0/24。

			支持 IPv6 地址匹配：::1，fe80::1，fe80::1/64 等。
remote_addrss	多个不同 IP 地址	["127.0.0.1", "192.0.0.0/8", "::1"]	
methods	请求方式	["GET", "POST"]	一个或多个的组合：GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS, CONNECT, TRACE, PURGE 如果为空或没有该选项，代表没有任何 method 限制
priority	匹配优先级	例：priority = 10	如果不同路由包含相同 uri，根据属性 priority 确定哪个 route 被优先匹配， 值越大优先级越高，默认值为 0
labels	标识附加属性的键值对	{"version": "v2", "build": "16", "env": "production"}	
timeout	设置 upstream 的连接、发送消息、接收消息的超时时间（单位为秒）	{"connect": 3, "send": 3, "read": 3}	这个配置将会覆盖在 upstream 中 配置的 timeout 选项
enable_websocket	是否启用 websocket(boolean)		缺省 false。
status	是否启用此路由		1 表示启用，0 表示禁用 缺省 1
create_time	创建时间戳，单位为秒		如果不指定则自动创建
update_time	更新时间戳，单位为秒		如果不指定则自动创建
vars	参数匹配	["arg_name", "==", "json"] ["arg_age", ">", 18]	由一个或多个[var, operator, val]元素组成的列表。 ["arg_name", "==", "json"], 表示当前请求参数 name 是 json。 这里的 var 与 Nginx 内部自身变量命名是保持一致，所以也可以使用 request_uri、host 等。

		当获取 cookie 的值时，cookie name 是区分大小写字母的。 例如：var 等于 "cookie_x_foo" 与 var 等于 "cookie_X_Foo" 表示不同的 cookie。 例：<code>[["arg_name", "=", "json"], ["arg_age", ">", 18]]</code>
--	--	---

VARS 支持的操作符

操作符	解释	示例
==	equal	<code>{"arg_name", "=", "json"}</code>
~=	not equal	<code>{"arg_name", "~=", "json"}</code>
>	greater than	<code>{"arg_age", ">", 24}</code>
<	less than	<code>{"arg_age", "<", 24}</code>
~~	Regular match	<code>{"arg_name", "~~", "[a-z]+"}</code>
~*	Case insensitive regular match	<code>{"arg_name", "~*", "[a-z]+"}</code>
in	find in array	<code>{"arg_name", "in", {"1", "2"}}</code>
has	left value array has value in the right	<code>{"graphql_root_fields", "has", "repo"}</code>
!	reverse the result	<code>{"arg_name", "!", "~~", "[a-z]+"}</code>
ipmatch	ip address match	<code>{"remote_addr", "ipmatch", {"127.0.0.1", "192.168.0.0/16"}}</code>